



Sinhgad Institutes

DEPARTMENT OF COMPUTER ENGINEERING
RMD SINHGAD SCHOOL OF ENGINEERING
WARJE, PUNE – 411058

CERTIFICATE

This is to certify that the project report entitled

“Online Food Ordering System”

Submitted by

Name: Adil Hamid Khan

PRN No: 72336713C

is a bona fide work carried out by them under the supervision of **Ms. Manisha Desai**. And it is submitted towards the partial fulfilment of the requirement of Savitribai Phule Pune University for Third Year.

Ms. Manisha Desai
Guide,
Department of Computer Engineering

Dr. Deepali Newaskar
Head,
Department of Computer Engineering

Dr. V. V. Dixit
Principal,
RMD Sinhgad School of Engineering Pune – 58

Place: PUNE
Date:

I

Certificate by Guide

This is to certify that **Mr. Adil Hamid Khan** has completed the MINI Project work under my guidance and supervision and that, I have verified the work for its originality in documentation, problem statement, implementation and results presented in the Project. Any reproduction of other necessary work is with the prior permission and has been given due ownership and included in the references.

Place: PUNE

Date:

Signature of Guide
(**Ms. Manisha Desai**)

II

ACKNOWLEDGEMENT

It is our pleasure to acknowledge sense of gratitude to all those who helped us in making this project.

We thank our Mini Project Guide **Ms. Manisha Desai** for helping us and providing all necessary information regarding our project.

We are also thankful to **Dr. Deepali Newaskar (Head - Department of Computer Engineering)** for providing us the required facilities and helping us while carrying out this project work.

Finally, we wish to thank all our teachers and friends for their constructive comments, suggestions and criticism and all those directly or indirectly helped us in completing this project.

.

NAME OF THE STUDENT

Adil Hamid Khan

III

ABSTRACT

The Online Food Ordering System is a web-based application designed to simplify the process of ordering food from restaurants. This system allows users to browse menus, select food items, add them to a cart, and place orders efficiently from any location. It provides a convenient and user-friendly interface that enhances the overall customer experience.

The system is designed using a modular approach by applying software architecture principles and design patterns. The Model-View-Controller (MVC) architecture is used to separate the application into different components such as data management, user interface, and business logic. Additional design patterns such as Singleton and Factory are incorporated to improve system scalability, maintainability, and flexibility.

The application includes features like user authentication, restaurant and menu management, cart handling, and order processing. The system ensures efficient data handling and smooth interaction between components.

Overall, this project demonstrates how software architecture and design patterns can be effectively used to develop a structured and scalable application. The Online Food Ordering System can be extended further for real-world deployment with additional features like online payment integration and real-time tracking.

IV

CONTENTS

Certificate	
Certificate By Guide	I
Acknowledgement	II
Abstract	III
1. Introduction	1
1.1 Objectives	
1.2 Problem Statement	
1.3 Project Scope	
2. System Architecture	3
3. System Requirement	7
3.1 Hardware Requirements	
3.2 Software Requirements	
4. Diagrams	8
5. Coding	10
6. Result and Discussion	15
7. Advantages and Disadvantages	17
8. Conclusion	18
9. References	19

SR. NO.	NAME OF FIGURES	PAGE NO:
1	SYSTEM ARCHITECTURE	3
2	USECASE Diagram	4
3	Data Flow Diagram	8
4	CLASS DIAGRAM	8
5.	SEQUENTIAL DIAGRAM	9
6.	DEPLOYMENT DIAGRAM	9

Introduction

In today's fast-paced digital world, the demand for online services has increased significantly, especially in the food industry. Online Food Ordering Systems have become an essential part of modern lifestyle, allowing users to order food conveniently from their homes without visiting restaurants. These systems provide an efficient platform for customers to browse menus, select food items, and place orders in a quick and user-friendly manner.

The proposed system aims to develop an Online Food Ordering System that simplifies the process of ordering food and improves the interaction between customers and restaurants. The system focuses on providing a structured and scalable solution by applying software architecture principles and design patterns.

1.1 Objectives

The main objective of this project is to design and develop an efficient Online Food Ordering System that provides a seamless experience for users to browse food items, place orders, and manage transactions. The system aims to implement a well-structured architecture using the Model-View-Controller (MVC) pattern to ensure separation of concerns and improve maintainability.

Additionally, the project focuses on incorporating design patterns such as Singleton and Factory to enhance flexibility and reusability. The system is intended to provide features like user authentication, menu management, cart handling, and order processing, thereby ensuring smooth and efficient functioning of the application.

1.2 Problem Statement

The increasing demand for quick and convenient food ordering services has highlighted the limitations of traditional ordering methods such as phone calls or in-person visits. These methods are often time-consuming, prone to errors, and lack proper management of orders and customer data.

The problem addressed in this project is to develop an Online Food Ordering System that enables users to easily browse menus, select food items, and place orders in an efficient and organized manner.

The system should ensure proper handling of user data, order processing, and communication between different components while maintaining scalability and performance.

1.3 Project Scope

The scope of this project includes the design and development of a basic Online Food Ordering System that allows users to register, log in, browse available food items, add items to a cart, and place orders. The system also includes functionalities for managing user information, order details, and basic menu data.

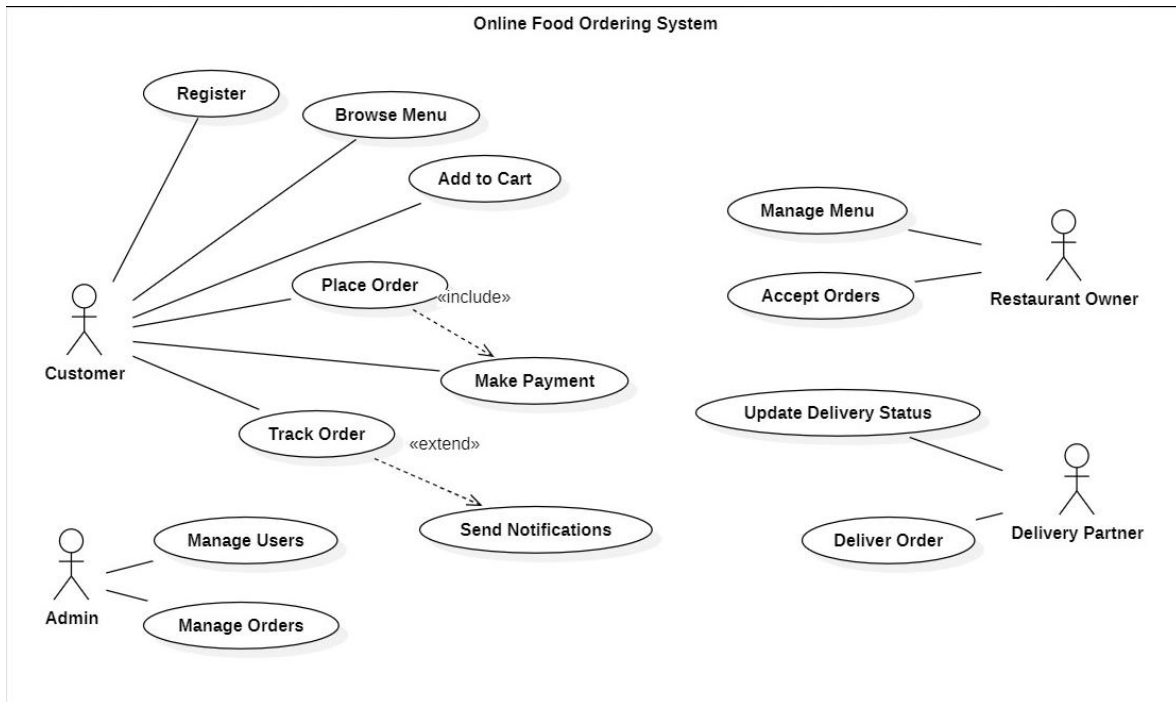
The project primarily focuses on demonstrating software architecture and design patterns rather than building a fully deployed commercial application. Advanced features such as online payment integration, real-time delivery tracking, and third-party API integration are considered out of scope but can be added in future enhancements. The system is designed to be scalable and can be extended to support additional features and larger user bases.

Authentication & Authorization



TE- DEPARTMENT OF COMPUTER ENGINEERING

Use Case Diagram



Use Cases

- **User Registration & Login:**

Users can create an account and log in to the system. Authentication ensures secure access.

- **Browse Menu:**

Customers can view available restaurants and food items along with prices and details.

- **Add to Cart:**

Customers can select food items and add them to their cart for ordering.

- **Place Order:**

Customers can place an order after confirming cart items and delivery details.

- **Payment Processing:**

System integrates with payment gateway to process online payments securely.

- **Order Management:**

Admin and restaurant owners can manage orders, update status, and handle requests.

- **Order Tracking:**

Customers and delivery partners can track order status in real-time.

- **Delivery Management:**

Delivery partners receive orders and update delivery status.

- **Notification System:**

System sends notifications for order confirmation, status updates, and delivery.

- **Database Interaction:**

System stores and retrieves user, order, and menu data efficiently.

- **Authentication & Authorization:**

Ensures only valid users can access system functionalities.

System Requirements

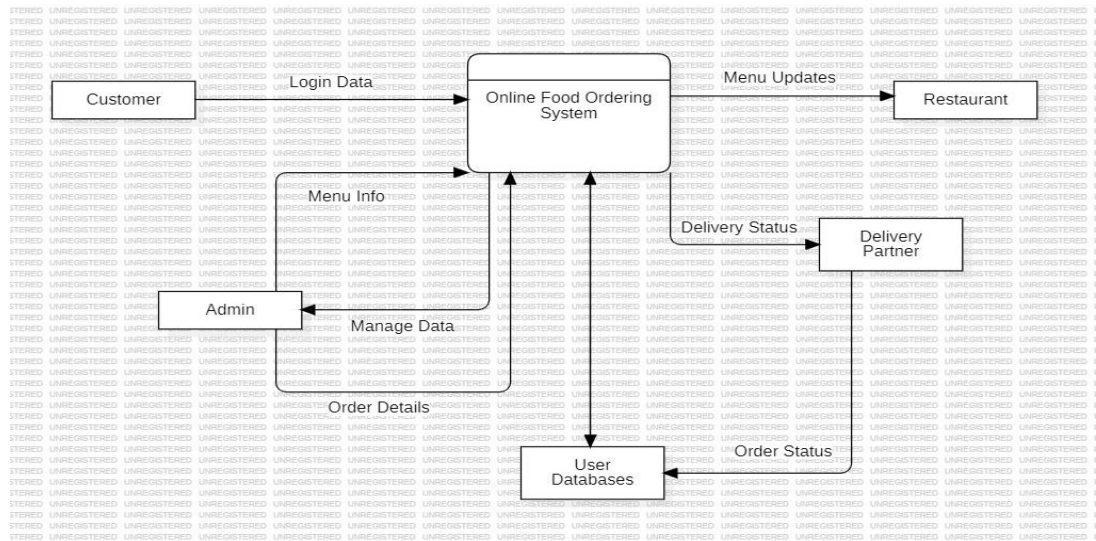
Hardware Requirements:

- ✓ **Processor:** Intel i3 or higher (i5 recommended for better performance)
- ✓ **RAM:** Minimum 4 GB (8 GB recommended for handling multiple users)
- ✓ **Storage:** At least 500 MB free disk space
- ✓ **System Type:** 64-bit computer or laptop
- ✓ **Network:** Stable internet connection for communication between users and server
- ✓ **Input Devices:** Keyboard and Mouse

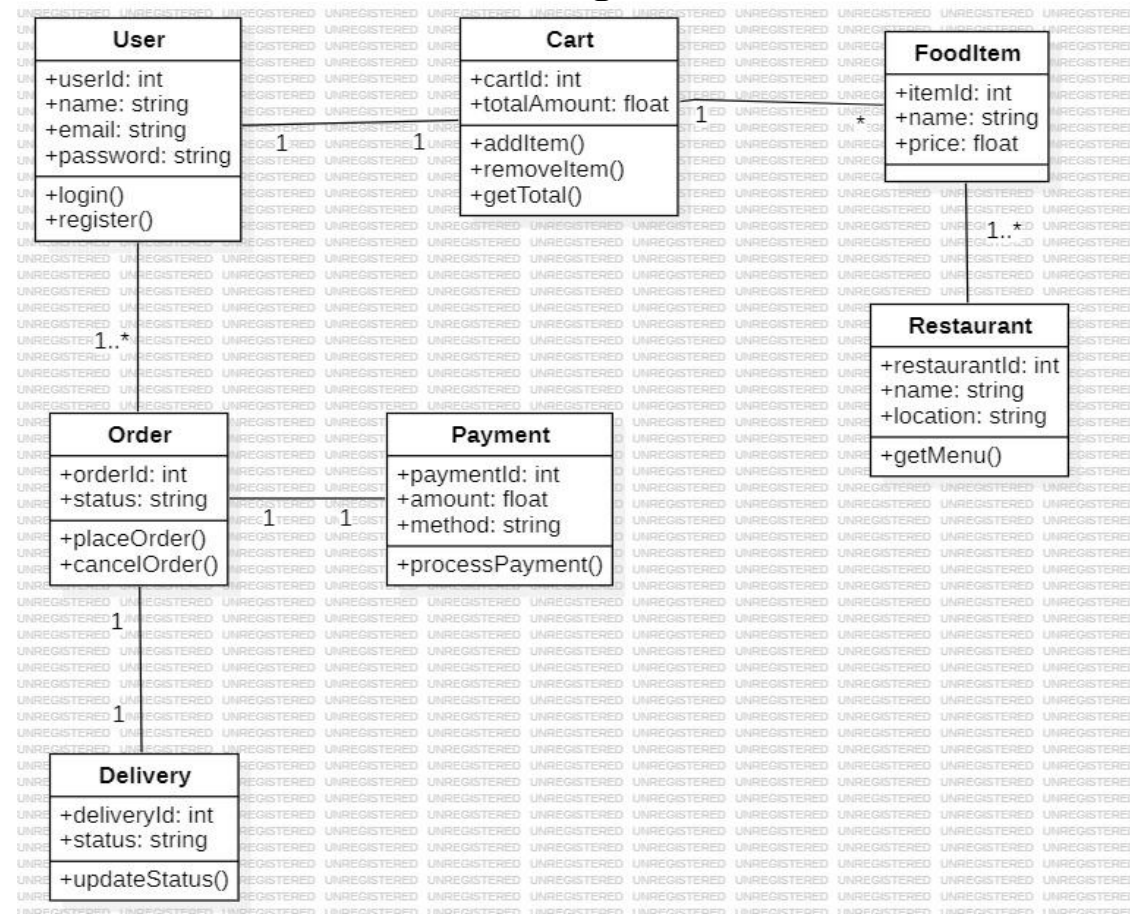
Software Requirements:

- ✓ **Operating System:** Windows 10/11, Linux (Ubuntu), or macOS
- ✓ **Programming Language:** Python / Java / PHP
- ✓ **Frontend Technologies:** HTML5, CSS3, JavaScript
- ✓ **Backend Framework:** Flask / Django / Node.js (optional)
- ✓ **Database Management System:** MySQL / SQLite
- ✓ **Web Server:** Apache / Localhost (XAMPP/WAMP)
- ✓ **Development Tools:** VS Code / PyCharm
- ✓ **Browser:** Google Chrome / Microsoft Edge
- ✓ **Version Control (Optional):** Git

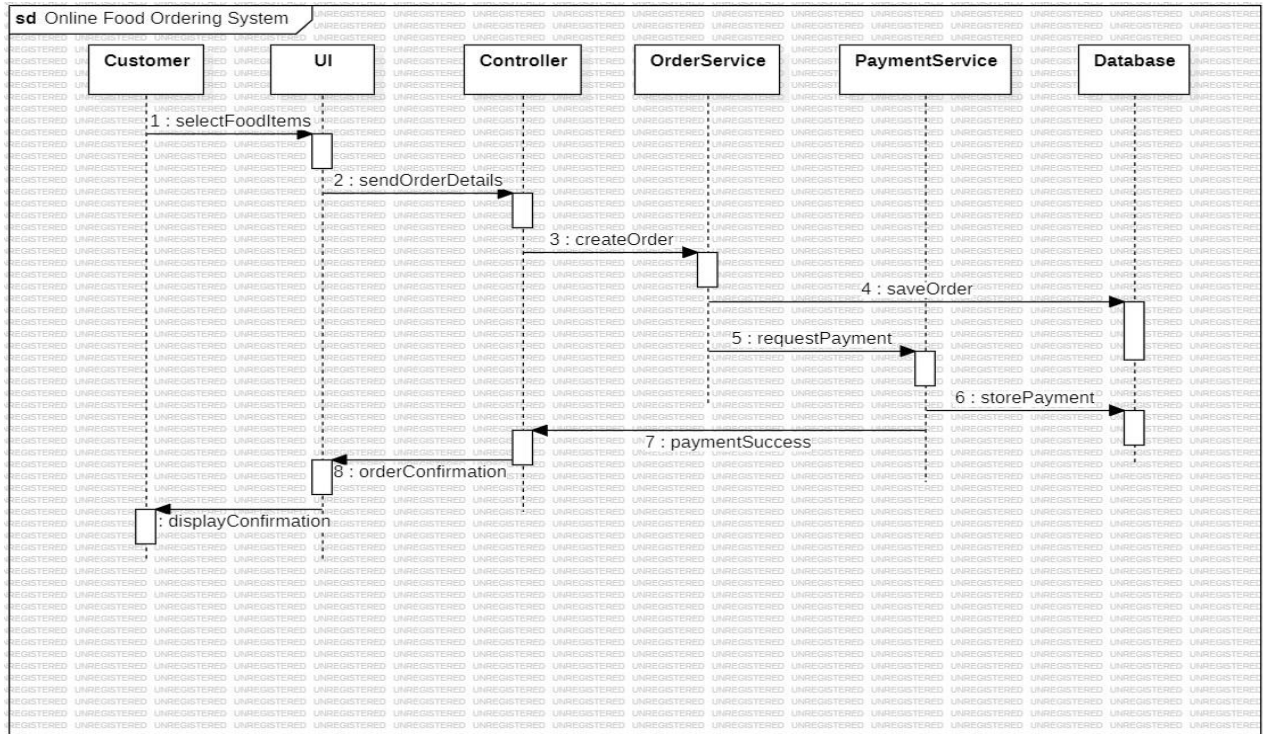
Data Flow Diagram



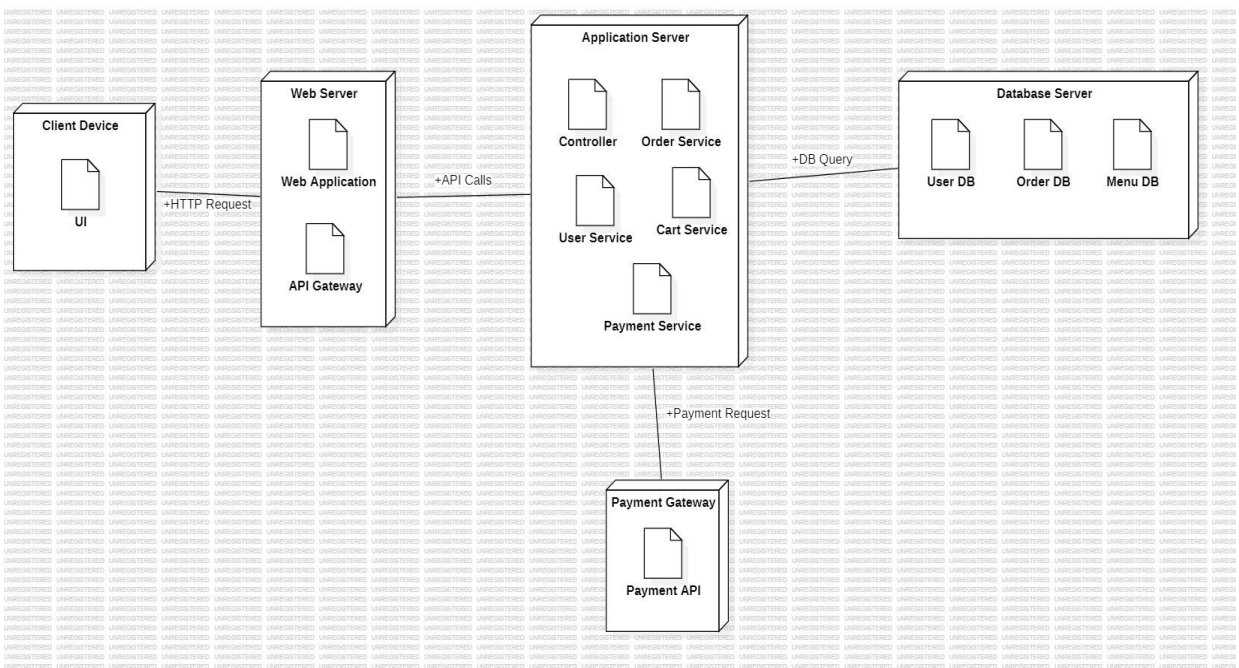
Class Diagram



Sequence Diagram



Deployment Diagram



Coding

app.py

```
from flask import Flask, render_template, redirect, url_for

app = Flask(__name__)

menu = [

    {"id": 1, "name": "Pizza", "price": 200, "img": "https://source.unsplash.com/300x200/?pizza"},

    {"id": 2, "name": "Burger", "price": 120, "img":
"https://source.unsplash.com/300x200/?burger"},

    {"id": 3, "name": "Pasta", "price": 150, "img": "https://source.unsplash.com/300x200/?pasta"},

    {"id": 4, "name": "Fries", "price": 90, "img": "https://source.unsplash.com/300x200/?fries"}

]

cart = []

@app.route('/')

def index():

    return render_template('index.html', menu=menu)

@app.route('/add/<int:id>')

def add(id):

    for item in menu:

        if item["id"] == id:

            cart.append(item)

    return redirect(url_for('cart_view'))

@app.route('/cart')
```



```

def cart_view():
    total = sum(item['price'] for item in cart)
    return render_template('cart.html', cart=cart, total=total)

@app.route('/order')
def order():
    cart.clear()
    return render_template('order.html')

if __name__ == '__main__':
    app.run(debug=True)

```

Templates/index.html

```

{% extends "base.html" %}

{% block content %}

<h1>Explore Menu</h1>

<div class="grid">
    {% for item in menu %}
    <div class="card">
        
        <h3>{{ item.name }}</h3>
        <p>₹{{ item.price }}</p>
        <a href="/add/{{ item.id }}">Add to Cart</a>
    
```

```
</div>

{% endfor %}

</div>
```

```
{% endblock %}
```

Templates/base.html

```
<!DOCTYPE html>

<html>

<head>

    <title>FoodHub</title>

    <link rel="stylesheet" href="/static/style.css">

</head>

<body>

<div id="loader">Loading...</div>

<nav>

    <h2> FoodHub</h2>

    <div>

        <a href="/">Home</a>

        <a href="/cart">Cart</a>

        <span class="toggle-btn" onclick="toggleTheme()"> </span>

    </div>

</nav>
```

```

<div class="content">

    {% block content %} {% endblock %}

</div>

<script>

function toggleTheme() {

    document.body.classList.toggle("light");

}

window.onload = function() {

    document.getElementById("loader").style.display = "none";

}

</script>

</body>

</html>

```

Templates/cart.html

```

{% extends "base.html" %}

{% block content %}

<h1>Your Cart</h1>

<div class="cart">

```

```

    {% for item in cart %}

    <div class="cart-item">

        <p>{{ item.name }}</p>

        <span>₹{{ item.price }}</span>

    </div>

    {% endfor %}

</div>


<h2>Total: ₹{{ total }}</h2>


<a class="btn" href="/order">Place Order</a>


{% endblock %}

```

Templates/order.html

```

{% extends "base.html" %}

{% block content %}


<div class="success">

    <h1> Order Placed Successfully!</h1>

    <p>Your food is on the way </p>

    <a href="/">Back to Menu</a>

</div>


{% endblock %}

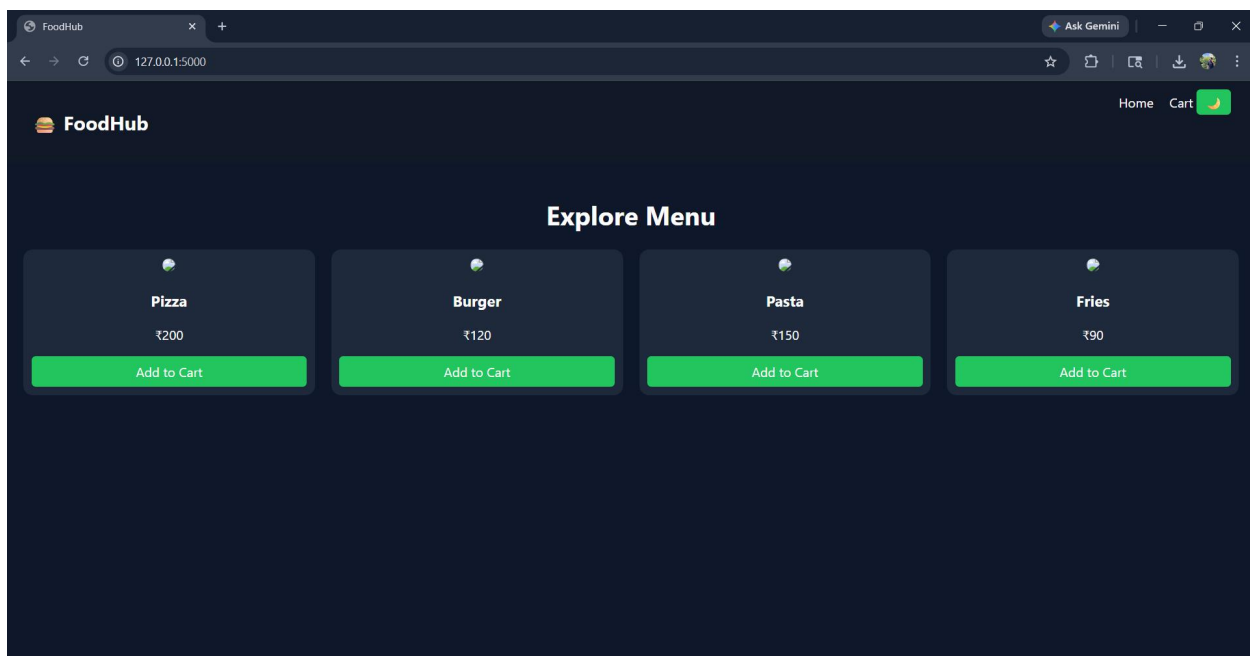
```

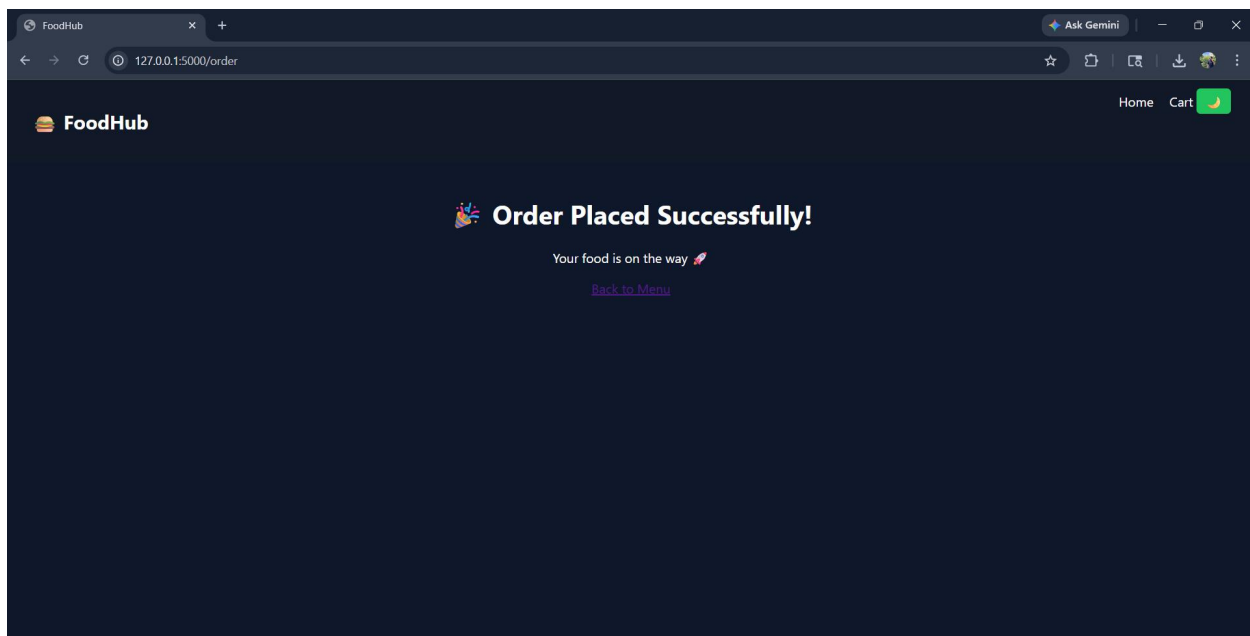
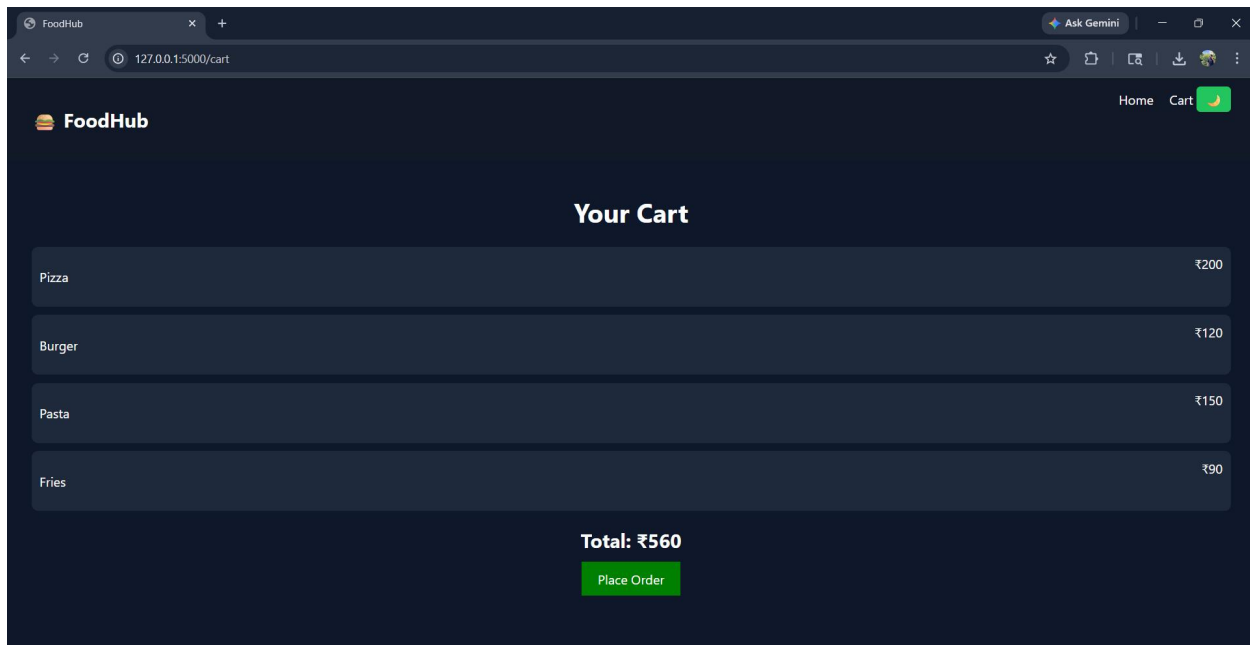
Result and Discussion

The developed Online Food Ordering System was successfully implemented as a responsive web application using the MVC architecture. The system provides a user-friendly interface where customers can browse menus, add items to the cart, and place orders seamlessly across devices (mobile, tablet, and desktop). The separation of concerns in MVC ensured better code organization, making the system modular, scalable, and easy to maintain.

The Model efficiently handled data operations such as user details, menu items, and order processing, while the View provided a dynamic and responsive UI. The Controller acted as an intermediary, ensuring smooth communication between user inputs and back-end logic. The application demonstrated fast response time and smooth navigation.

Overall, the use of MVC improved development efficiency, enhanced maintainability, and provided a clean architecture. The output screens validated correct functionality, including order placement, user interaction, and real-time updates.





Advantages and Disadvantages

Advantages	Disadvantages
Modular Architecture: MVC structure separates logic, UI, and data, making the system easy to maintain.	Modular Architecture: MVC structure separates logic, UI, and data, making the system easy to maintain.
Responsive Design: The application works smoothly across mobile, tablet, and desktop devices.	Responsive Design: The application works smoothly across mobile, tablet, and desktop devices.
User-Friendly Interface: Simple navigation and interactive UI enhance overall user experience.	User-Friendly Interface: Simple navigation and interactive UI enhance overall user experience.
Scalability: The system can be easily extended with features like database, login, and payment integration.	Scalability: The system can be easily extended with features like database, login, and payment integration.

Conclusion

The Online Food Ordering System was successfully designed and implemented using a structured software engineering approach with the Model-View-Controller (MVC) architecture. The project effectively demonstrates how modern web applications can be developed with clear separation of concerns, resulting in improved maintainability, scalability, and code organization. The system provides essential functionalities such as browsing food items, managing a cart, and placing orders through a responsive and user-friendly interface.

The implementation highlights the practical application of software architecture principles and design patterns in building real-world systems. The responsive design ensures accessibility across various devices, enhancing user experience and usability. Additionally, the modular structure of the application allows for easy future enhancements, such as integration with databases, authentication systems, and online payment gateways.

Overall, this project serves as a strong foundation for understanding web development, system design, and architectural patterns. It successfully meets the defined objectives and demonstrates the importance of structured design in developing efficient and scalable software solutions.

References

- ✓ Flask Documentation
<https://flask.palletsprojects.com/>
- ✓ MDN Web Docs
<https://developer.mozilla.org/>
- ✓ W3Schools
<https://www.w3schools.com/>
- ✓ Bootstrap Documentation
<https://getbootstrap.com/>
- ✓ GeeksforGeeks – MVC Architecture
<https://www.geeksforgeeks.org/mvc-design-pattern/>
- ✓ Tutorialspoint – Software Architecture
https://www.tutorialspoint.com/software_architecture_design/
- ✓ Unsplash
<https://unsplash.com/>
- ✓ Stack Overflow
<https://stackoverflow.com/>